SHELFSENSE AI

Planogram Designer MVP

Stage 1 Product Specification & Build Acceptance Criteria

Version 0.1 | Prepared for Antigravity implementation | 20 October 2026

STAGE BOUNDARY: This is a local-first planogram editor. It creates and saves reference layouts for later AI shelf-compliance work. Computer vision, YOLO/CLIP, shelf-label detection, OCR and AI compliance scoring are explicitly out of scope for this first increment.

1. Product intent

ShelfSense AI will let a retail user create a reference planogram inside the product rather than requiring an uploaded planogram. The user creates shelf objects with real measurements, adds catalogue products, places them across shelf rows, saves the layout locally, and exports the visual and structured layout for use by a future shelf-image audit pipeline.

2. Reference images and visual direction

Reference screenshot	Use it for
Retail Shelf Planner	Visual convention for multiple shelf rows and product packs arranged across a shelf.
Formal planogram sheet	Measurement arrows, row/shelf information, product placement and exportable reference layout.
MoEngage dashboard	Dark charcoal app shell, sidebar, restrained accent color, clean cards, hierarchy and professional spacing. Do not copy its content.
Blender Outliner	Nested object tree: Planogram > Shelves > Rows > Product placements; selectable and expandable groups.

The outer application can use a dark analytics-style shell, but the editor canvas must be bright white/light-grey with dark labels so the shelf and measurement annotations are easy to inspect. Do not stretch product images; preserve aspect ratio and size them from physical dimensions.

3. Scope: exactly two primary pages

Page	Required behavior
Dashboard	Real local-data KPIs: saved planograms, catalogue products, shelves; recent plans with name/updated time/shelf count; Create and Open actions; clear empty state; no fabricated analytics.

Page	Required behavior
Design Your Own Planogram	Initially empty bright canvas; Add Shelf; Outliner tree; shelf canvas with measurement labels; right Properties/Product Library panel; product form; drag-and-drop; save/reopen; capacity feedback; visual and JSON exports.

4. Core data and editing behavior

Product catalogue

- Unique SKU and required product name.
- Product image with preview; support JPEG, PNG and WebP uploads with size/type validation.
- Width, height and depth in centimetres; positive numeric values.
- Non-negative price and explicit currency (display INR/₹ by default).
- Products appear in the catalogue after saving and persist across refresh and backend restart.
- Search by SKU/name is required; category filtering is a useful extra.
- Seed at least three products, preferably all six supplied sample SVG items. Their dimensions and prices are synthetic demo values.

Shelf and placement

- Create multiple independent shelves; no shelf exists until Add Shelf is clicked.
- Shelf properties: name, width, total height, depth, row count (1–12), optional base/plinth and its height.
- Render an outliner hierarchy for planogram, shelves, rows and product placements.
- Drag catalogue items to shelf rows; dragging again adds a separate facing; allow move/reorder and removal.
- Store row index and horizontal offset in centimetres, not screen pixels.
- Reject overlap and items that exceed usable row width; explain the issue rather than shrinking products.
- Show row used-width/capacity and measurements in the editor.
- Changing row count must never silently remove placements. Confirm or block destructive edits.

Persistence and export

- Use SQLite through the backend as the source of truth; store uploaded images as local files and DB paths as metadata.
- Save a named planogram and reopen it from Dashboard. Saved state must survive browser refresh and application restart.
- Visual export must show shelf structure, product placements, names/SKUs where readable and measurements in cm. PNG required; SVG is acceptable as an additional/fallback format.
- JSON export must contain format_version, units, planogram name/ID, shelves, shelf dimensions, row information, product SKU/dimensions and placement offsets in cm.

5. Technical approach

Layer	Recommended choice
Frontend	Next.js / React / TypeScript / Tailwind CSS
Drag and drop	@dnd-kit or an equivalent maintained library
Backend	FastAPI + Pydantic + SQLAlchemy
Local persistence	SQLite; uploaded image files in a configurable local media folder

Layer	Recommended choice
Backend tests	pytest + FastAPI TestClient / HTTPX
Frontend tests	Vitest + React Testing Library
Browser flow	Playwright for the save/reload/export critical journey

Keep a small monorepo and modular monolith. Do not add cloud hosting, authentication, queues, YOLO/CLIP/OCR, microservices or a vector database in this stage. Honor the existing repository state if an intentional compatible stack is already present.

6. Minimum data model

Entity	Essential fields
Product	id, sku, name, category, image_path, width_cm, height_cm, depth_cm, price, currency, timestamps
Planogram	id, name, timestamps
Shelf	id, planogram_id, name, width_cm, height_cm, depth_cm, row_count, plinth_enabled, plinth_height_cm
Placement	id, shelf_id, row_index, product_id, x_offset_cm, order/position metadata

Rows may be normalized or derived if the implementation preserves stable row indices and correctly handles row-count changes. Use foreign keys and service-level validation. Pixel positions must never be the only persisted positions.

7. Suggested API

Expose equivalent tested CRUD operations even if the final route names differ: GET /health; product list/create/read/update/delete; planogram list/create/read/update/delete; shelf create/update/delete; placement create/update/delete; planogram JSON export. Document final routes and schemas in docs/API_CONTRACT.md. Return clear 4xx errors for duplicate SKUs, invalid dimensions, and out-of-bounds placements.

8. Folder structure

```
Tredence_H4ck/
├── AGENTS.md / README.md / .gitignore
├── PRODUCT_SPEC.md / USER_GUIDE.md / TEST_PLAN.md
├── ANTIGRAVITY_IMPLEMENTATION_PROMPT.md
├── sample_assets/ (catalogue + images)
├── docs/ (API_CONTRACT.md, TESTING.md)
├── backend/
│   ├── app/ (main.py, api/, core/, db/, services/, storage/)
│   ├── tests/ (unit/, integration/)
│   └── pyproject.toml
├── frontend/
│   ├── src/ (app/, components/, features/, lib/, types/)
│   ├── tests/ and package.json
└── e2e/ (Playwright critical flow)
```

Keep folders proportionate to the implementation. Avoid empty abstraction layers. Keep sample assets in Git; ignore the runtime SQLite database, user-uploaded files, secrets and generated build outputs.

9. Required testing

Test level	Minimum coverage
Backend unit	Product/shelf validation; positive dimensions; valid row count; plinth constraints; width/capacity math; accepted fit; rejected overflow/overlap; row movement; stable JSON fields.
Backend integration	Product upload/create/read persistence; planogram+shelf+placement flow; invalid placement error; update/delete behavior; JSON export; isolated temporary DB and uploads.
Frontend component	Empty shelf state; Add Shelf; product form validation/preview; measured properties; catalogue contents; placement success/error; saved/failed states; real dashboard counts.
Playwright E2E	Open app -> create shelf -> set dimensions/rows -> place product -> save -> reload -> verify persistence -> export JSON and visual planogram.

Tests must not depend on downloading models or external APIs. The coding agent must actually run checks, report exact outcomes and label blocked/not-run tests honestly. Never test against the developer's persistent SQLite database.

10. Acceptance checklist

- Two primary pages only: Dashboard and Design Your Own Planogram.
- Blank first-use canvas; Add Shelf creates selectable object in outliner and canvas.
- Create/edit product image, name, unique SKU, measurements and price; persisted after refresh/restart.
- Multiple shelves and custom width/height/depth/rows/plinth behave correctly.
- Drag/drop placement uses cm-based fit rules and prevents overlap/out-of-bounds.
- Dashboard reads real local DB state; saved plans can be reopened.
- Visual PNG (or documented SVG) and versioned JSON exports include dimensions and placements.
- Unit tests, integration tests and one end-to-end journey exist and are run.
- Local setup/use/testing documentation is complete; no secrets, runtime DB or uploads committed.

11. Out of scope

- YOLO/RT-DETR object detection; CLIP/image embeddings; custom model training.
- Shelf-label detection, OCR, photo-to-planogram comparison, AI compliance scoring or recommendations.
- Authentication, cloud database, remote deployment, multi-user synchronization and microservices.
- Full 3D CAD/Blender editing; Blender screenshot is a reference for the hierarchy panel only.

12. Implementation sequence and expected handoff

1. App shell, health endpoint and SQLite configuration.
2. Product catalogue CRUD, image handling and sample seed data.
3. Planogram/shelf CRUD and dimension/row editor.
4. Shelf canvas and outliner.
5. Drag/drop placement, capacity and overlap validation.
6. Save/reopen plus real-data Dashboard.
7. Visual and JSON exports.
8. Unit/integration/E2E tests, user guide, UI polish and regression verification.

Antigravity should inspect the repository, present a plan, then work in small vertical slices. It must not commit or push. Its final report must include the actual changed-file summary, run commands, test/build results, known limitations and next recommended task.

Appendix: bundled sample products

SKU	Sample product	Width × Height × Depth	Price
DEMO-DRK-001	Citrus Sparkling Drink	6.5 × 12.2 × 6.5 cm	₹35.00
DEMO-DRK-002	Orange Juice Carton	6.0 × 19.0 × 5.0 cm	₹60.00
DEMO-FOD-001	Oat Crunch Cereal	8.0 × 22.0 × 4.5 cm	₹120.00
DEMO-SNK-001	Crispy Chips	14.0 × 20.0 × 5.0 cm	₹30.00
DEMO-HOM-001	Fresh Laundry Liquid	9.0 × 25.0 × 6.0 cm	₹185.00
DEMO-HOM-002	Daily Clean Spray	7.0 × 24.0 × 5.0 cm	₹140.00

All products above use original unbranded vector artwork included in the handoff bundle. Dimensions and prices are fabricated for UI/testing and must be replaced with verified values for any real retail use.